

Memoria del proyecto: Robot de reparto autonomo

Asignatura: Inteligencia Ambiental

Equipo: Grupo F

Curso: 2025/2026

Plataforma: LEGO Mindstorms EV3 con Pybricks/MicroPython, aplicacion web y MQTT

1. Introduccion

El proyecto consiste en el diseno, construccion y programacion de un robot autonomo de reparto basado en LEGO Mindstorms EV3. El robot debe desplazarse por un mapa urbano formado por calles, cruces y edificios, recoger paquetes en puntos concretos y entregarlos en otros puntos indicados desde una aplicacion web.

El sistema completo combina tres partes principales: el robot fisico, una aplicacion web de control y visualizacion, y una comunicacion mediante MQTT. El robot recibe el mapa, calcula rutas entre puntos de recogida y entrega, sigue las lineas del circuito mediante el sensor de color y publica su estado y odometria para que la interfaz pueda mostrar su posicion en tiempo real.

Durante el desarrollo se ha trabajado especialmente en la navegacion real sobre el tablero, ya que el comportamiento del robot depende de factores fisicos como la posicion del sensor de color, el rozamiento de las ruedas, la precision de los giros, el peso de la pala y las diferencias entre girar a izquierda o derecha.

2. Objetivos

El objetivo principal es construir un sistema de reparto autonomo capaz de recibir pedidos, planificar una ruta y desplazarse por el mapa hasta completar la recogida y la entrega de un paquete.

Objetivos especificos:

- Interpretar el mapa de la ciudad recibido como una cadena codificada.
- Representar el mapa como un grafo de casillas conectadas.
- Calcular rutas validas entre dos puntos mediante busqueda en anchura.
- Controlar el movimiento del robot EV3 usando motores diferenciales.
- Detectar y seguir la linea verde del circuito mediante sensor RGB.
- Detectar cuadrados negros para identificar centros de casilla e intersecciones.
- Gestionar los giros de 90 y 180 grados teniendo en cuenta la calibracion real del robot.
- Permitir el envio de pedidos desde una aplicacion web.
- Comunicar robot e interfaz mediante MQTT.
- Publicar odometria y estado del robot durante la ejecucion.
- Documentar el despliegue para que el sistema pueda ser probado por el profesor.

3. Requisitos

3.1 Requisitos funcionales

- El robot debe recibir o cargar un mapa de la ciudad.
- La aplicacion web debe mostrar el mapa y permitir seleccionar puntos de recogida y entrega.
- El robot debe recibir pedidos mediante MQTT.
- El robot debe calcular la ruta mas corta entre su posicion actual y el destino.
- El robot debe navegar bloque a bloque hasta completar cada pedido.
- El robot debe recoger el paquete bajando la pala.
- El robot debe entregar el paquete subiendo la pala.
- El robot debe publicar su estado: esperando, navegando, recogiendo, entregando, completado o error.
- El robot debe publicar odometria de forma periodica.

3.2 Requisitos no funcionales

- El sistema debe poder ejecutarse en el EV3 con MicroPython/Pybricks.
- La comunicacion debe ser ligera y compatible con la red local.
- La interfaz debe ser usable desde un navegador sin instalacion compleja.
- El codigo debe estar dividido en modulos para facilitar mantenimiento y pruebas.
- La navegacion debe ser tolerante a pequenas desviaciones fisicas del robot.
- La configuracion de IP del broker debe poder actualizarse facilmente.

3.3 Restricciones

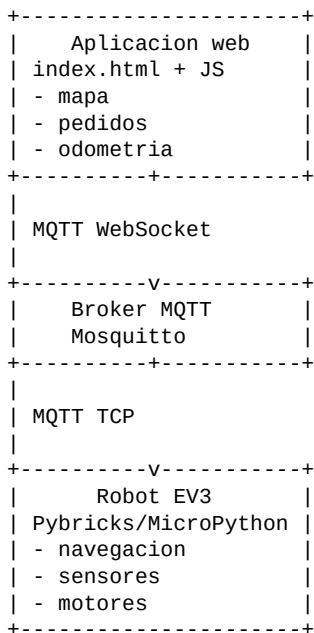
- El robot utiliza un unico sensor de color, situado delante y a la izquierda del centro del robot.
- Los giros no son perfectamente simetricos entre izquierda y derecha.
- El tablero contiene zonas con varias lineas cercanas, por lo que una recalibracion excesiva puede hacer que el robot detecte una linea incorrecta.
- La deteccion de negro y verde depende de las condiciones de luz y de la calibracion del sensor.

4. Arquitectura

El sistema se organiza en tres capas:

1. Robot EV3: ejecuta el control fisico, la planificacion de rutas, la navegacion y la comunicacion MQTT.
2. Broker MQTT: actua como intermediario entre la aplicacion web y el robot.
3. Aplicacion web: muestra el mapa, permite enviar pedidos y visualiza el estado del robot.

Arquitectura general:



4.1 Modulos del robot

- robot/main.py: punto de entrada. Inicializa hardware, MQTT, mapa y bucle principal.
- robot/hardware.py: abstraccion de motores, sensores, pala, pantalla, sonido y odometria.
- robot/map_parser.py: convierte la cadena del mapa en una matriz y un grafo de conexiones.
- robot/navigation.py: calcula rutas, gestiona orientacion, giros y seguimiento de linea.
- robot/mqtt_client.py: implementa el cliente MQTT del robot.

4.2 Modulos de la aplicacion web

- webapp/index.html: estructura de la interfaz.
- webapp/styles.css: estilos visuales.
- webapp/app.js: conexion MQTT, pedidos, estado y odometria.
- webapp/map_renderer.js: renderizado del mapa en canvas.
- webapp/images/: imagenes de los bloques del mapa.

4.3 Topics MQTT

Topic | Direccion | Uso
map | Web/broker -> robot | Envio del mapa codificado
Equipo F/pedidos | Web -> robot | Pedidos con recogida y entrega
Equipo F/posicion_inicial | Web -> robot | Configuracion de posicion inicial
Equipo F/odometria | Robot -> web | Posicion, orientacion y movimiento
Equipo F/estado | Robot -> web | Estado actual del pedido

5. Planificacion

El trabajo se organizo por fases:

Fase	Trabajo realizado	Resultado
1	Analisis del enunciado y mapa	Identificacion de bloques, calles y puntos de pedido
2	Construccion del robot	Chasis con dos motores, pala y sensor de color
3	Comunicacion MQTT	Envio de mapa, pedidos, estado y odometria
4	Aplicacion web	Interfaz para mapa, pedidos y seguimiento
5	Planificacion de rutas	Parser de mapa y BFS sobre grafo
6	Navegacion fisica	Seguimiento de linea verde y deteccion de cuadrados negros
7	Calibracion	Ajuste de giros, umbrales de color y salida de intersecciones
8	Pruebas finales	Validacion de entregas completas sobre el tablero

La fase mas iterativa fue la calibracion. Aunque el algoritmo de ruta funcionaba de forma logica, el comportamiento real del robot exigio ajustar parametros de giro, avance y busqueda de linea.

6. Diseno

6.1 Representacion del mapa

El mapa se recibe como una cadena de caracteres donde cada par de digitos representa un bloque. El mapa tiene 7 filas y 5 columnas. Cada ID indica que tipo de bloque es y que direcciones permite.

Ejemplos:

ID	Tipo
00	Edificio
01	Calle izquierda-derecha
02	Calle arriba-abajo
03	Curva arriba-derecha
11	Cruce con las cuatro direcciones

El modulo map_parser.py convierte estos IDs en una matriz y despues construye un grafo de adyacencia. Solo se conecta una casilla con otra si ambas tienen direcciones compatibles.

6.2 Calculo de rutas

Para calcular la ruta entre dos puntos se utiliza busqueda en anchura (BFS). Esta tecnica es adecuada porque todas las conexiones entre casillas tienen el mismo coste, por lo que BFS encuentra la ruta con menor numero de pasos.

Flujo del calculo de ruta:

```
Inicio
|
v
Leer posicion actual y destino
|
v
Ejecutar BFS sobre el grafo del mapa
|
v
Hay ruta-
|--- No ---> Mostrar error
|
Si
|
v
Convertir posiciones a direcciones
|
v
Ejecutar movimiento bloque a bloque
```

6.3 Navegacion bloque a bloque

Cada tramo de la ruta se ejecuta en dos pasos:

1. Girar hasta la direccion necesaria.
2. Avanzar hasta el siguiente bloque siguiendo la linea verde.

El robot mantiene una orientacion logica (up, right, down, left) y la convierte a angulos absolutos: 0, 90, 180 y 270 grados.

6.4 Seguimiento de linea verde

El sensor de color lee valores RGB. Para detectar la linea verde se calcula:

```
verdosidad = G - max(R, B)
```

Si la verdosidad supera el umbral configurado, se considera que el robot esta sobre la linea verde. El seguimiento usa un controlador proporcional-derivativo:

```
error = LINE_THRESHOLD - verdosidad  
giro = Kp * error + Kd * (error - error_anterior)
```

Esto permite que el robot corrija su trayectoria mientras avanza.

6.5 Deteccion de cuadrados negros

Los cuadrados negros indican centros de casilla e intersecciones. El robot los detecta combinando dos condiciones:

```
intensidad = R + G + B  
es_negro = intensidad < BLACK_INTENSITY_THRESHOLD  
y verdosidad < LINE_THRESHOLD
```

La segunda condicion evita confundir una linea verde oscura con negro.

6.6 Maquina de estados del avance

El seguimiento de linea utiliza una maquina de estados para distinguir entre salir del centro actual, cruzar la frontera y llegar al centro siguiente.

```
Fase 0: salir del cuadrado negro actual  
|  
v  
Fase 1: seguir linea verde hasta detectar frontera negra  
|  
v  
Fase 2: cruzar frontera negra hasta volver a verde  
|  
v  
Fase 3: buscar el cuadrado negro central del siguiente bloque  
|  
v  
Parar y centrar ruedas
```

6.7 Tratamiento especial de giros

El robot real no gira exactamente lo mismo que el valor pedido por software. Por ello se han calibrado factores de correccion:

Parametro	Valor actual	Funcion
TURN_90_CORRECTION	1.60	Compensa giros de 90 grados
TURN_180_CORRECTION	1.25	Compensa giros de 180 grados
RIGHT_TURN_REDUCTION	0.60	Reduce giros a derecha para no pasarse
RIGHT_TURN_LINE_SEARCH_DISTANCE	90 mm	Avance corto tras girar a la derecha

Tambien se decidio que los giros de 180 grados se hagan siempre hacia la izquierda, porque en las pruebas resultaban mas fiables que los giros de 180 hacia la derecha.

La posicion real del sensor condiciona esta parte: el sensor esta situado delante y a la izquierda del centro del robot. Por este motivo, tras ciertos giros a la derecha el sensor podia quedar sobre el negro o detectar una linea incorrecta. La solucion final fue reducir el giro derecho y avanzar un poco buscando verde antes de activar la busqueda en abanico.

6.8 Búsqueda de línea

Si el robot sale del negro y no encuentra la línea verde, realiza una búsqueda en abanico. Esta búsqueda gira sobre el sitio, sin avanzar, para evitar que el robot invada líneas cercanas y detecte una calle incorrecta.

```
No hay verde
|
v
Girar a derecha buscando verde
|
v
Girar a izquierda cruzando el centro
|
v
Volver al centro
|
v
Repetir con abanico algo mayor
```

7. Implementacion

El proyecto esta implementado principalmente en Python/MicroPython para el robot y JavaScript para la interfaz web.

7.1 Codigo del robot

El archivo main.py realiza la inicializacion general:

- Inicializa motores, sensores y pantalla.
- Conecta con el broker MQTT.
- Espera el mapa.
- Crea el navegador.
- Entra en un bucle de espera de pedidos.
- Ejecuta la ruta de recogida y despues la ruta de entrega.

El archivo hardware.py encapsula el acceso fisico al EV3:

- Motores izquierdo y derecho.
- Motor de pala.
- Sensor de color.
- Sensor giroscopico.
- DriveBase para movimiento diferencial.
- Pantalla, luz y sonido.

El archivo navigation.py contiene la parte principal de autonomia:

- Búsqueda de rutas con BFS.
- Conversion de rutas a direcciones.
- Gestion de orientacion.
- Giro calibrado.
- Seguimiento de línea.
- Búsqueda de línea en abanico.
- Recogida y entrega de paquete.

7.2 Codigo de la interfaz web

La aplicacion web permite:

- Conectarse al broker MQTT mediante WebSockets.
- Recibir y representar el mapa.
- Configurar la posicion inicial.
- Crear pedidos de recogida y entrega.
- Mostrar la cola de pedidos.
- Visualizar odometria y estado del robot.

El mapa se dibuja en un elemento canvas. Las imagenes de bloques permiten que la representacion sea mas parecida al tablero real.

7.3 Comunicacion MQTT

La comunicacion se basa en mensajes JSON simples. Un pedido tiene esta estructura:

```
{
"pickup": [2, 1],
"delivery": [5, 0]
}
```

Un mensaje de odometria incluye informacion como:

```
{
"row": 4,
"col": 0,
"heading": "up",
"heading_angle": 0,
"distance": 125,
"speed": 80,
"angle": 0,
"turn_rate": 5
}
```

8. Despliegue

8.1 Requisitos de software

- Visual Studio Code.
- Extension LEGO MINDSTORMS EV3 MicroPython.
- Extension ev3dev-browser.
- Mosquitto MQTT Broker.
- Navegador web moderno.
- Python en el ordenador para ejecutar set_ip.py.

8.2 Configuracion de red

El ordenador y el robot deben estar conectados a la misma red Wi-Fi. La IP del ordenador se configura en los archivos del robot y de la web mediante:

```
python set_ip.py <IP_DEL_ORDENADOR>
```

Este script actualiza:

- webapp/app.js
- robot/mqtt_client.py

8.3 Arranque del broker MQTT

Desde la raiz del proyecto:

```
& "C:\Program Files\mosquitto\mosquitto.exe" -v -c mosquitto.conf
```

La terminal debe permanecer abierta mientras se prueba el sistema.

8.4 Ejecucion del robot

1. Conectar el EV3 al ordenador mediante USB o Wi-Fi.
2. Abrir el proyecto en Visual Studio Code.
3. Subir la carpeta robot/ al EV3.
4. Ejecutar main.py desde el entorno EV3.
5. Esperar a que el robot muestre que esta listo o esperando mapa.

8.5 Ejecucion de la web

1. Abrir webapp/index.html en el navegador.
2. Comprobar que aparece conexion al broker.
3. Esperar o reenviar el mapa.
4. Seleccionar punto de recogida y entrega.
5. Enviar el pedido.

9. Resultados

El sistema final consigue completar rutas de recogida y entrega usando el mapa, la planificacion BFS y el seguimiento de linea verde.

9.1 Parametros finales de navegacion

Parametro	Valor	Observacion
LINE_THRESHOLD	20	Umbral de verdosidad
BLACK_INTENSITY_THRESHOLD	45	Umbral de negro
LINE_SPEED	80 mm/s	Velocidad de seguimiento
PROPORTIONAL_GAIN	3.5	Ganancia proporcional
DERIVATIVE_GAIN	2.0	Ganancia derivativa
TURN_90_CORRECTION	1.60	Correccion de giro de 90
TURN_180_CORRECTION	1.25	Correccion de giro de 180
RIGHT_TURN_REDUCTION	0.60	Ajuste especifico de giro derecho
RIGHT_TURN_LINE_SEARCH_DISTANCE	90 mm	Avance tras giro derecho

9.2 Pruebas realizadas

Prueba	Resultado	Comentario
Recepcion de mapa por MQTT	Correcta	El robot recibe la cadena y la parsea
Envio de pedido desde web	Correcto	El pedido llega al topic del equipo
Calculo de ruta	Correcto	BFS encuentra rutas validas
Seguimiento de linea verde	Correcto tras calibracion	Requiere umbrales ajustados
Deteccion de cuadrados negros	Correcta	Se evita confundir verde oscuro con negro
Giro a derecha	Correcto tras ajuste	Se redujo el giro y se anadio busqueda corta de linea
Giro de 180 grados	Correcto	Se fuerza hacia la izquierda
Publicacion de odometria	Correcta	La web actualiza posicion y estado

9.3 Imagenes y videos

Incluir aqui evidencias de funcionamiento:

- Imagen del robot montado: imagenes/robot.jpg
- Imagen del tablero o ciudad: ciudad.png
- Captura de la aplicacion web funcionando: imagenes/webapp.png
- Video de recogida y entrega: enlace pendiente
- Video de seguimiento de linea y giro en intersecciones: enlace pendiente

10. Mejoras futuras

- Anadir una segunda lectura de color o un segundo sensor para detectar mejor la posicion lateral de la linea.
- Usar el giroscopio de forma mas intensiva para cerrar el control de giro.
- Guardar estadisticas automaticas de cada pedido: tiempo total, distancia recorrida, numero de realineaciones y errores.
- Mejorar la interfaz para mostrar la ruta planificada antes de ejecutarla.
- Implementar replanificacion si el robot falla al llegar a un bloque.
- Crear un modo de calibracion guiado desde la web para ajustar umbrales sin modificar codigo.
- Reducir la dependencia de parametros empiricos mediante control mas robusto.
- Anadir confirmacion visual de recogida y entrega mediante sensores o deteccion de carga.

11. Referencias

- Documentacion de Pybricks para EV3: <https://docs.pybricks.com/>
- Documentacion de LEGO Mindstorms EV3.
- Documentacion de MQTT: <https://mqtt.org/>
- Eclipse Mosquitto: <https://mosquitto.org/>
- Material de la asignatura Inteligencia Ambiental.
- Codigo fuente del proyecto del Grupo F.